SQL INJECTION ATTACK & PREVENTION

A Project Report Submitted to

JNTU, Ananthapuramu

In partial fulfilment of the requirements for the award of the degree of

Bachelor of technology

(Computer Science & Engineering)

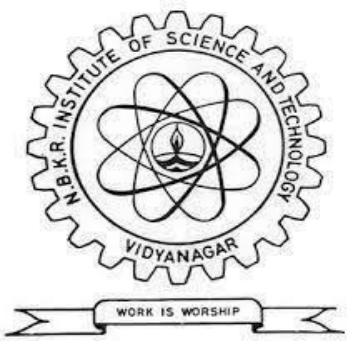
By

P. SAI VARDHAN	-	19KB1A05C0
I. GANESH	-	20KB5A0507
A. VISHNU	-	20KB5A0509
S. SURYA	-	20KB5A0512

Under the esteemed Guidance of

Mrs. B. RAJANI, M.Tech.,(Ph.D)

Assistant Professor, Dept. of CSE



DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

N.B.K.R. INSTITUTE OF SCIENCE & TECHNOLOGY

(AUTONOMOUS)

VIDYANAGAR – 524 413, TIRUPATHI DIST, AP

APRIL - 2023

Website: www.nbkrist.org Ph. No.: 08624-228 247

Email: ist@nbkrist.org Fax : 08624-228 257

N.B.K.R. INSTITUTE OF SCIENCE & TECHNOLOGY

(Autonomous)

(Approved by AICTE: Accredited by NBA: Affiliated to JNTU, Ananthapuramu)

An ISO 9001-2000 Certified Institution

Vidyanagar-524413, Tirupathi District, Andhra Pradesh, India

BONAFIDE CERTIFICATE

This is to Certify that the major project work entitled "**SQL Injection Attack And Prevention**" is a bonafide work done by **P. SAI VARDHAN – 19KB1A05C0, I. GANESH – 20KB5A0507, A.VISHNU - 20KB5A0509, S.SURYA – 20KB5A0512** in the department of **Computer Science & Engineering. N.B.K. R Institute of Science and Technology, Vidyanagar** and is submitted to JNTU, Ananthapuramu in the partial fulfilment for the award of B. Tech degree in Computer Science & Engineering. This work has been carried out under my supervision.

Mrs. B. RAJANI
Asst. professor
Department of CSE
NBKRIST, Vidyanagar

Dr A. Rajasekhar Reddy
Professor & Head
Department of CSE
NBKRIST, Vidyanagar

Submitted for the Viva-Voce Examination held on _____

Internal Examiner **External Examiner**

ACKNOWLEDGEMENT

The satisfaction that accompanies the successful completion of a project would be incomplete without the people who made it possible of their consistent guidance and encouragement crowned our efforts with success.

We would like to express my profound sense of gratitude to our project guide **Mrs. B. RAJANI, Department of Computer Science & Engineering, N.B.K.R Institute of Science & Technology (Affiliated to JNTUA, Ananthapuramu), Vidyanagar**, for her masterful guidance and the constant encouragement throughout the project. My sincere appreciations for her suggestions and unmatched services without, which this work would have been an unfulfilled dream.

We convey our special thanks to **Dr. Y. VENKATA RAMI REDDY,** respectable chairman of **N.B.K.R. Institute of Science and Technology**, for providing excellent infrastructure in our campus for the completion of the project.

We convey our special thanks to **Sri. N. RAM KUMAR REDDY, respectable Correspondent** of **N.B.K.R. Institute of Science and Technology**, for providing excellent infrastructure in our campus for the completion of the project.

We are grateful to **Dr. V VIJAYA KUMAR REDDY, Director, N.B.K.R. Institute of Science and Technology** for allowing us to avail all the facilities in the college.

We express our sincere gratitude to **Dr. A. RAJA SEKHAR REDDY, Professor & Head of Department, Computer Science & Engineering**, for providing exceptional facilities for successful completion of our project work.

We would like to convey our heartful thanks to **Staff Members, Lab Technicians, and my friends**, who extend their cooperation in making this project as a successful one.

We would like to thank one and all who have helped me directly and indirectly to complete this project successfully.

ABSTRACT

SQL injection is a type of cyber-attack where malicious SQL statements are inserted into an application's entry fields, allowing attackers to access sensitive data or control the application's behavior. The aim of a SQL prevention project is to implement measures and strategies to protect a software application from SQL injection attacks. This involves identifying vulnerabilities in the application's code, implementing secure coding practices, and deploying tools and technologies to prevent and detect attacks. The ultimate goal is to ensure that sensitive data and system resources are protected from unauthorized access, maintaining the confidentiality, integrity, and availability of the application and its data. The project may also involve educating developers and users on the risks of SQL injection and best practices for preventing such attacks.

Table of Contents

CHAPTER 1: INTRODUCTION

1.1 Introduction

SQL injection is a type of attack that occurs when a malicious user inserts code into a web application that uses SQL databases. This code can modify, delete, or retrieve data from the database, depending on the intent of the attacker. SQL injection attacks are typically carried out by entering special characters or commands into the application's input fields, such as login forms or search bars.

The main reason why SQL injection attacks are so prevalent is that many web applications do not properly sanitize user input. This means that when a user enters a value into an input field, the application does not check to ensure that it is valid or safe. If the input is not validated properly, the attacker can use it to insert their own SQL code into the application's database query.

For example, suppose a web application allows users to search for products by entering a product name in a search bar. If the application does not validate the user's input.

This input might look like a legitimate product name, but it is actually an SQL command that tells the application to retrieve all data from the database, regardless of any filters or restrictions. The semicolon terminates the current SQL command, and the double dash indicates that the rest of the line should be treated as a comment, effectively nullifying any subsequent SQL code.

Once an attacker has successfully executed an SQL injection attack, they can gain access to sensitive data, such as usernames, passwords, or credit card numbers. They can also modify or delete data, potentially causing irreparable harm to the application and its users.

To prevent SQL injection attacks, web application developers need to ensure that all user input is properly sanitized and validated. This involves checking input values for special characters or commands that might indicate an SQL injection attack, such as apostrophes, semicolons, or double dashes. Developers can also use parameterized queries or prepared statements to separate user input from the SQL code and prevent attacks.

In conclusion, SQL injection attacks are a serious threat to the security of web applications that use SQL databases. They can result in the theft or modification of sensitive data and can cause significant damage to the application and its users. By following best practices for input validation and SQL query construction, developers can help prevent SQL injection attacks and ensure the safety of their applications.

1.2 Motivation

There are several compelling motivations for doing an SQL injection prevention project:

Security: SQL injection attacks are one of the most common and damaging types of attacks that web applications face. By implementing effective SQL injection prevention measures, web developers can significantly improve the security of their applications and protect sensitive user data.

Reputation: A successful SQL injection attack can result in a significant loss of trust from users and stakeholders. By preventing these attacks, developers can safeguard their organization's reputation and maintain the confidence of their users.

Compliance: Many industries and regions have strict data protection regulations that require organizations to take measures to protect sensitive data. By implementing effective SQL injection prevention measures, organizations can ensure compliance with these regulations and avoid legal and financial penalties.

Performance: SQL injection attacks can cause significant performance issues for web applications, including slow response times and downtime. By preventing these attacks, developers can improve the performance and reliability of their applications.

Professional development: Developing an SQL injection prevention project can be an excellent opportunity for developers to gain experience in cybersecurity and improve their skills in database design and development. It can also demonstrate their commitment to security best practices and enhance their professional reputation.

1.3 Objective

The objectives of an SQL injection prevention project may include:

Identifying vulnerabilities: The project should aim to identify any existing SQL injection vulnerabilities in the application's code and database structure.

Developing prevention measures: The project should involve developing and implementing effective prevention measures, such as input validation and parameterized queries.

Testing and validation: The project should include thorough testing and validation to ensure that the prevention measures are effective and do not cause any unintended consequences or performance issues.

Education and awareness: The project should aim to educate developers and other stakeholders about the importance of SQL injection prevention and the potential consequences of an attack.

Compliance: If the project is being undertaken to comply with data protection regulations, the objective should be to ensure that the prevention measures meet the relevant standards and requirements.

Ongoing maintenance: The project should include a plan for ongoing maintenance and monitoring to ensure that the prevention measures remain effective and up-to-date with emerging threats and best practices.

Overall, the objective of an SQL injection prevention project should be to reduce the risk of SQL injection attacks and protect the integrity and confidentiality of the application's data. By achieving this objective, developers can enhance the security and reliability of their applications and maintain the trust and confidence of their users and stakeholders.

CHAPTER 2: LITERATURE REVIEW

2.1 Literature Survey

SQL injection is a well-known and prevalent attack vector that targets web applications. This attack involves injecting malicious SQL code into an application's database, potentially allowing an attacker to access sensitive data, modify or delete data, or even take over the entire system. The following literature survey provides an overview of the research on SQL injection, including its prevalence, detection, prevention, and mitigation techniques.

Prevalence of SQL Injection:

Various studies have demonstrated the prevalence of SQL injection attacks. A study by Verizon revealed that SQL injection was one of the most common attack vectors in data breaches. Similarly, a report by Imperva found that SQL injection was the most common technique used by hackers to exploit web applications.

Detection of SQL Injection:

Several approaches have been proposed for detecting SQL injection attacks. These include static analysis, dynamic analysis, and hybrid approaches. Static analysis involves analysing the application's source code to detect vulnerable code patterns. Dynamic analysis involves monitoring the application's runtime behaviour to detect anomalies that could indicate an attack. Hybrid approaches combine both static and dynamic analysis techniques to improve detection accuracy.

Prevention of SQL Injection:

Preventing SQL injection attacks involves a combination of techniques, including input validation, parameterized queries, stored procedures, and web application firewalls. Input validation involves checking user input for potential SQL injection attacks. Parameterized queries involve using parameter placeholders to separate user input from the SQL query. Stored procedures involve predefining SQL queries that are called by the application. Web application firewalls sit between the application and the database and can detect and block SQL injection attacks.

Mitigation of SQL Injection:

In addition to prevention, mitigation techniques are also necessary to handle SQL injection attacks that bypass prevention measures. These techniques include log analysis, intrusion detection, and incident response. Log analysis involves analysing application and database logs to detect and investigate potential SQL injection attacks. Intrusion detection involves monitoring the network and application for signs of SQL injection attacks. Incident response involves developing a plan of action for responding to SQL injection attacks, including isolating affected systems, removing the attacker's access, and patching vulnerabilities.

2.2 Modules

There are different types of modules that can be used in SQL injection prevention. Here are some of the commonly used ones:

- **Input validation and sanitization modules**: These modules are used to validate and sanitize user input before it is processed by the application. They can be used to detect and remove malicious code from input data, thus preventing SQL injection attacks.
- **Database firewall modules:** These modules are used to provide an additional layer of security to the database. They can monitor incoming traffic to the database and block any requests that appear to be malicious.
- **Query analysis modules**: These modules are used to analyze the SQL queries that are being executed by the application. They can detect and block any queries that are suspected to be SQL injection attacks.
- **Stored procedure modules:** Stored procedures can be used to encapsulate SQL code and provide an additional layer of security. By using stored procedures, developers can ensure that only authorized users can execute certain SQL commands.
- **Encryption and hashing modules:** These modules can be used to encrypt sensitive data or hash passwords before storing them in the database. This can help prevent attackers from accessing sensitive information even if they manage to bypass other security measures.
- **Access control modules:** These modules can be used to restrict access to the database based on user roles and permissions. By controlling who can access the database, developers can prevent unauthorized users from executing malicious SQL queries.

Overall, using a combination of these modules can help provide robust protection against SQL injection attacks and other types of database vulnerabilities.

2.3 Existing System

There are several projects and tools available for preventing SQL injection attacks. Here are some examples:

1. **Parameterized queries:** One of the most effective ways to prevent SQL injection attacks is to use parameterized queries in your code. Parameterized queries use placeholders for user input, which are then bound to variables before the query is executed. This prevents malicious input from being executed as part of the SQL statement.

2. **Input validation:** Another approach is to validate user input before it is used in a SQL query. This can include checking the data type, length, and format of user input, as well as using regex to filter out malicious input.

3. **ORM frameworks:** Object-Relational Mapping (ORM) frameworks can also help prevent SQL injection attacks. ORM frameworks like Hibernate and Entity Framework use prepared statements and parameterized queries behind the scenes to prevent SQL injection.

4. **Web Application Firewalls (WAFs)**: WAFs can also help prevent SQL injection attacks by filtering out malicious input before it reaches your application. They do this by analysing incoming traffic and blocking requests that contain known SQL injection attack patterns.

5. **Code analysis tools:** There are several code analysis tools available that can scan your code for SQL injection vulnerabilities. These tools can help identify potential vulnerabilities and provide suggestions for how to fix them.

2.4 Proposed System

A web application is vulnerable to an SQL injection attack if an attacker is able to insert SQL statements into an existing SQL query of the application. This is usually achieved by injecting malicious input into user fields that are used to compose the query, login page prompts the user to enter their username and password into a form typically used for checking the user login credentials therefore, are prime targets for an attacker. In this example, if the login application does not perform correct input validation of the form fields, the attacker can inject strings into the query that alter its semantics. For example, consider a hacker entering "**OR 1=1" --** one of SQL injection string as in the table(1), the "--" command indicates a comment in Transact-SQL. Hence, everything after the first "--" is ignored by the SQL database engine with the help of the first quote in the input string, the user's name string is closed, while the "OR 1=1" adds a clause to the query which evaluates to true for every row in the table. When executing this query, the database returns all user rows, which applications often interpret as a valid login. So, in the proposed system all client-supplied data needs to be cleansed of any characters or strings that could possibly be used maliciously.

In this section list proposed methods can be applied to minimize the risk of a SQL injection attack .

Validation Input:

The first step in any form processing script should check the syntax of input for validity to verify that the input is a valid input in the language. The validation could be anything such as checking whether the entered value for "password" is a number, and is 16 or over, or making sure the username column has only valid characters such as A to Z, a to z and many classes of input have fixed languages such Email addresses, dates etc.

Encoding Input:

Sometimes input might contain some illegal characters, or it might not always be viable to validate all user input for example, in a search field the user could type anything that they are searching for, including script tags such as < script>, encoding is the best way to neutralize harmful data from user input, since encoding translates the harmful characters into their display equivalents for example the < character will be translated into < character. The HTML Encode method of the server object can be used to encode the harmful characters.

Filtering Input:

If an application does not filter user input before it is used in a SQL query, then SQL injection vulnerability occurs because server received incorrect input from an untrusted client. So proposed system prevent SQL injection attacks by filtering user input using language-supplied input filtering functions before using untrusted input in a SQL query. For example, let's suppose to filter out special characters such as the following: [] { } ; &. to achieve this functionality, use the Replace method of the String object this code will replace all the unwanted characters from the user's input.

Low privilege:

Limit database permissions and segregate users and never allows to connect as a database administrator in web application will limit the damage potential. The proposed system has three pages, the code of first page shown an HTML page with a form element. The form asks the visitor to enter his MySQL user name and password. The name of this page is login.html, when user full the data and click login button the data will send by post method to the second page that called login.php

2.5 Feasibility Study

A feasibility study on SQL injection prevention would assess the practicality and viability of implementing measures to prevent SQL injection attacks on a specific web application. Here are some factors that would be evaluated in a feasibility study:

Cost:

The cost of implementing SQL injection prevention measures, including software and hardware requirements, as well as training and support costs, would be evaluated. The cost would need to be justified in terms of the potential risk of a SQL injection attack and the value of the data being protected.

Technical requirements:

The technical requirements for implementing SQL injection prevention measures would be evaluated, including compatibility with existing systems and potential impact on system performance.

Resource availability:

The availability of resources, including personnel and equipment, would be assessed to ensure that the implementation of SQL injection prevention measures could be carried out effectively.

Security needs:

The level of security needed to protect the web application from SQL injection attacks would be evaluated, based on the sensitivity of the data being stored and the potential impact of a successful attack.

CHAPTER 3: SYSTEM REQUIRMENTS

3.1 Front-End Tools

- HTML
- CSS
- BOOTSTRAP

3.2 Back-End Tools

- PHP
- SQL

3.3 Software Requirements

- Windows 10 or higher
- XAMPP Apache tomcat server
- MySQL database

3.4 Hardware Requirements

- Processor - Core i3
- Hard Disk - 512GB
- Memory - 4GB RAM

CHAPTER 4: SYSTEM ANALYSIS

4.1 User Requirements

User requirements for SQL injection prevention will depend on the specific context and use case of the system being developed or deployed. However, here are some general user requirements for SQL injection prevention:

- **Input validation:** Users should be able to specify the types, formats, and constraints of data inputs that are accepted by the system. The system should validate input data against these specifications to ensure that it is valid and safe to use in SQL queries.

- **Parameterized queries:** Users should be able to create and execute SQL queries that use parameterized inputs, where input data is passed as parameters rather than being directly concatenated into the SQL query string.

- **Security configuration:** Users should be able to configure the security settings of the system to protect against SQL injection attacks. This may include specifying how incoming data should be sanitized, configuring WAFs and other security measures, and setting access control policies.

- **Monitoring and reporting:** Users should be able to monitor the system for SQL injection attacks and receive alerts when suspicious activity is detected. The system should also provide detailed reports on security events and vulnerabilities, including information on the type and severity of each incident.

- **Integration with other security tools:** Users should be able to integrate the system with other security tools and frameworks, such as intrusion detection and prevention systems, firewalls, and vulnerability scanners. This will help provide a comprehensive defence against SQL injection attacks.

- **User education and awareness:** Users should be educated on the importance of preventing SQL injection attacks and trained on how to identify and respond to potential security threats. This will help ensure that the system is used in a secure and responsible manner, and reduce the risk of SQL injection attacks caused by human error or negligence.

4.2 System Requirements

- ✓ **Operating System:** The project can be developed on a variety of operating systems, including Windows, Linux, or macOS.

- ✓ **Web Server:** A web server such as Apache or Nginx will be required to host the web application.

- ✓ **Programming Language**: The project can be implemented in a variety of programming languages, such as PHP, Python, or Java.

- ✓ **Database:** A relational database management system (RDBMS) such as MySQL, Oracle, or PostgreSQL will be required to store the application data.

- ✓ **Web Application Framework:** A web application framework such as Laravel, Django, or Spring can be used to help develop the web application.

- ✓ **Security Tools:** Tools such as web application firewalls, intrusion detection/prevention systems, and vulnerability scanners can be used to help prevent SQL injection attacks.

- ✓ **Hardware:** The system requirements for hardware will depend on the size and complexity of the web application. A basic configuration may include a standard desktop or laptop computer with at least 4GB of RAM and a dual-core processor.

4.3 Users

Users for SQL injection prevention can vary depending on the context and use case of the system being developed or deployed. Here are some examples of users who may be involved in SQL injection prevention:

- ❖ **Developers:** Developers are responsible for writing and testing code that prevents SQL injection attacks. They may use input validation, parameterized queries, and other security techniques to protect against SQL injection attacks.

- ❖ **Database administrators:** Database administrators are responsible for configuring and maintaining databases to prevent SQL injection attacks. They may configure access controls, monitor database activity, and implement security measures such as WAFs and intrusion detection systems.

- ❖ **Security professionals:** Security professionals are responsible for monitoring and managing overall system security. They may use tools such as vulnerability scanners, penetration testing, and code analysis to identify and prevent SQL injection attacks.

- ❖ **System administrators:** System administrators are responsible for configuring and maintaining server infrastructure to prevent SQL injection attacks. They may implement security measures such as firewalls, load balancers, and network segmentation to protect against SQL injection attacks.

- **End-users:** End-users are responsible for using the system in a secure and responsible manner. They may be trained on how to identify and respond to potential security threats such as SQL injection attacks, and may be required to follow security policies and best practices to prevent such attacks.

4.4 Features

The features of SQL injection prevention will depend on the specific context and use case of the system being developed or deployed. However, here are some general features that may be included in SQL injection prevention:

1. **Input validation:** The system should be able to validate input data against predefined specifications to ensure that it is valid and safe to use in SQL queries. This may involve using regular expressions or other techniques to filter out malicious input.

2. **Parameterized queries:** The system should support the use of parameterized queries, where input data is passed as parameters rather than being directly concatenated into the SQL query string. This will help prevent SQL injection attacks by ensuring that user input is properly sanitized and bound to variables.

3. **Sanitization:** The system should sanitize input data to remove any malicious code or characters that could be used to execute a SQL injection attack. This may involve using techniques such as whitelisting or blacklisting of characters or strings.

4. **Access controls:** The system should implement access controls to restrict access to sensitive data and functions. This will help prevent unauthorized users from accessing the database and executing SQL injection attacks.

5. **Encryption:** The system should use encryption to protect sensitive data in transit and at rest. This will help prevent attackers from intercepting or stealing sensitive data.

6. **Error handling:** The system should provide clear error messages to users when input data is rejected or when errors occur during query execution. This will help prevent attackers from using error messages to gain information about the database structure or data.

7. **WAFs:** The system may use Web Application Firewalls (WAFs) to inspect and filter incoming HTTP traffic for SQL injection attacks. WAFs can also help prevent other types of attacks such as cross-site scripting (XSS).

8. **Logging and monitoring:** The system should log all database activity and monitor for suspicious activity that may indicate a SQL injection attack. This will help detect and respond to attacks in real-time.

9. **User education and awareness:** The system should include user education and awareness features that help educate users on the importance of preventing SQL injection attacks and train them on how to identify and respond to potential security threats. This will help ensure that the system is used in a secure and responsible manner, and reduce the risk of SQL injection attacks caused by human error or negligence.

4.5 Data Modelling Diagrams

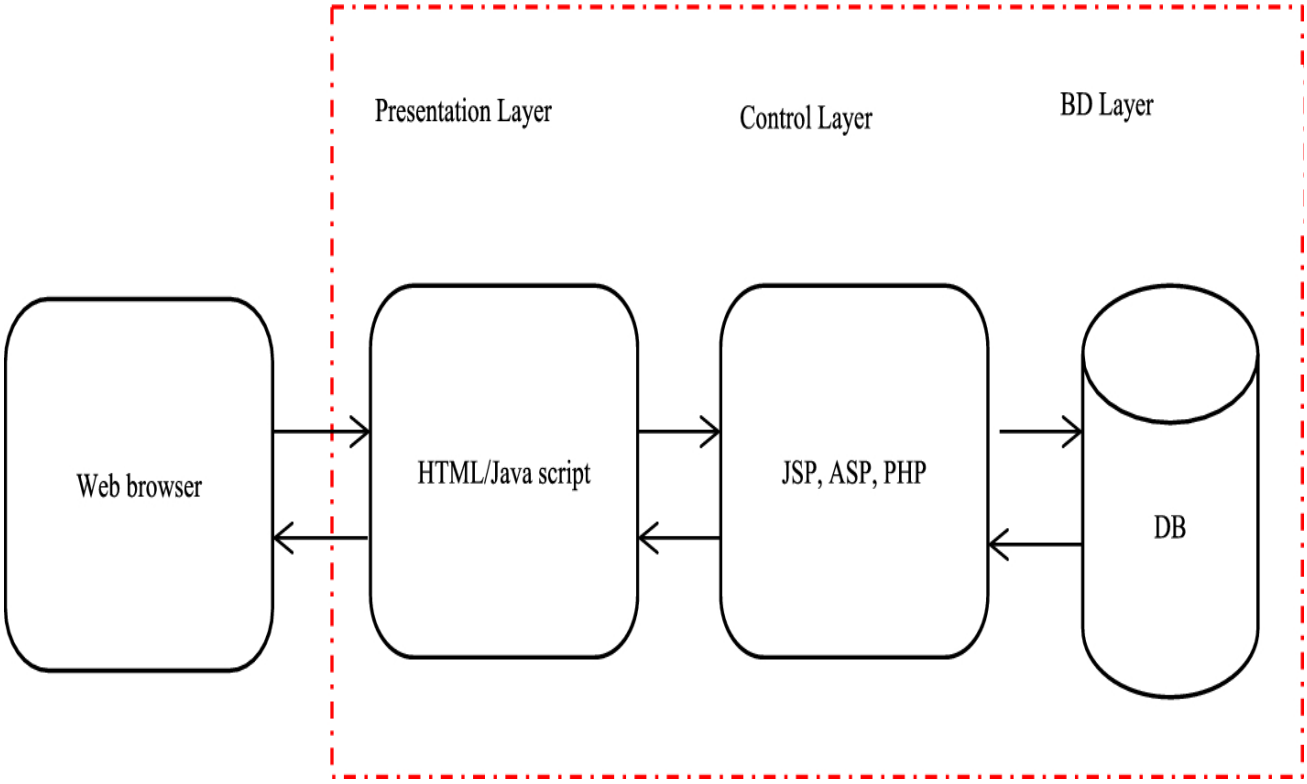


Fig: Web application Architecture

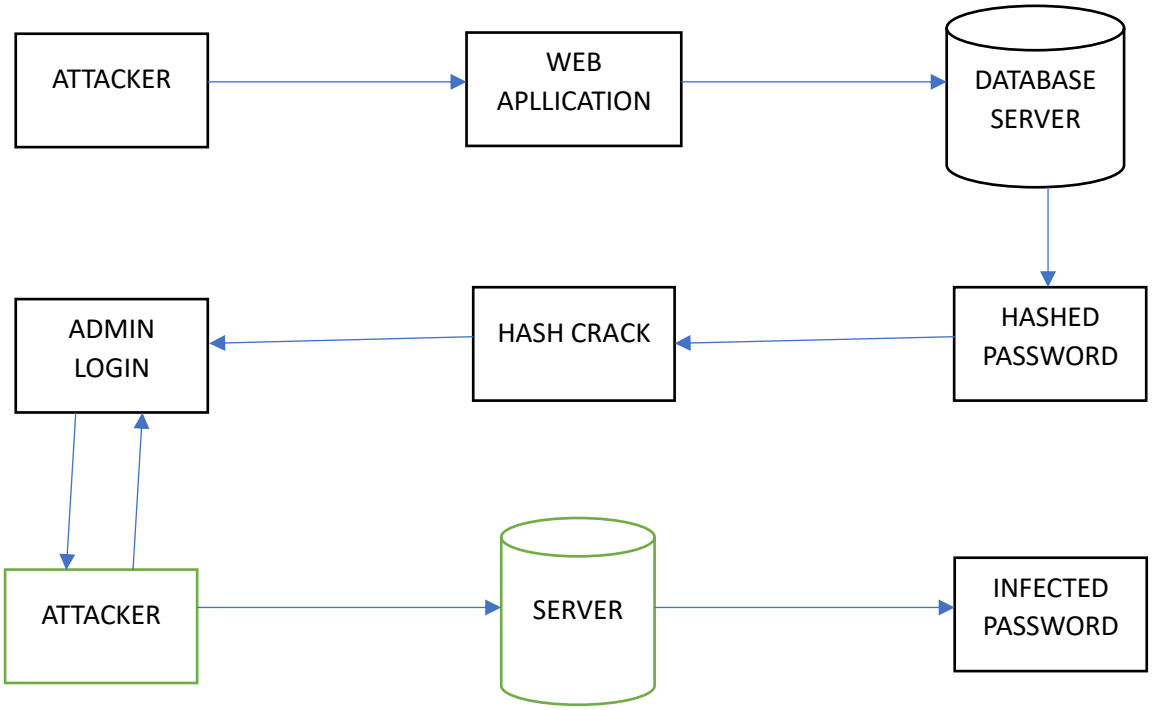
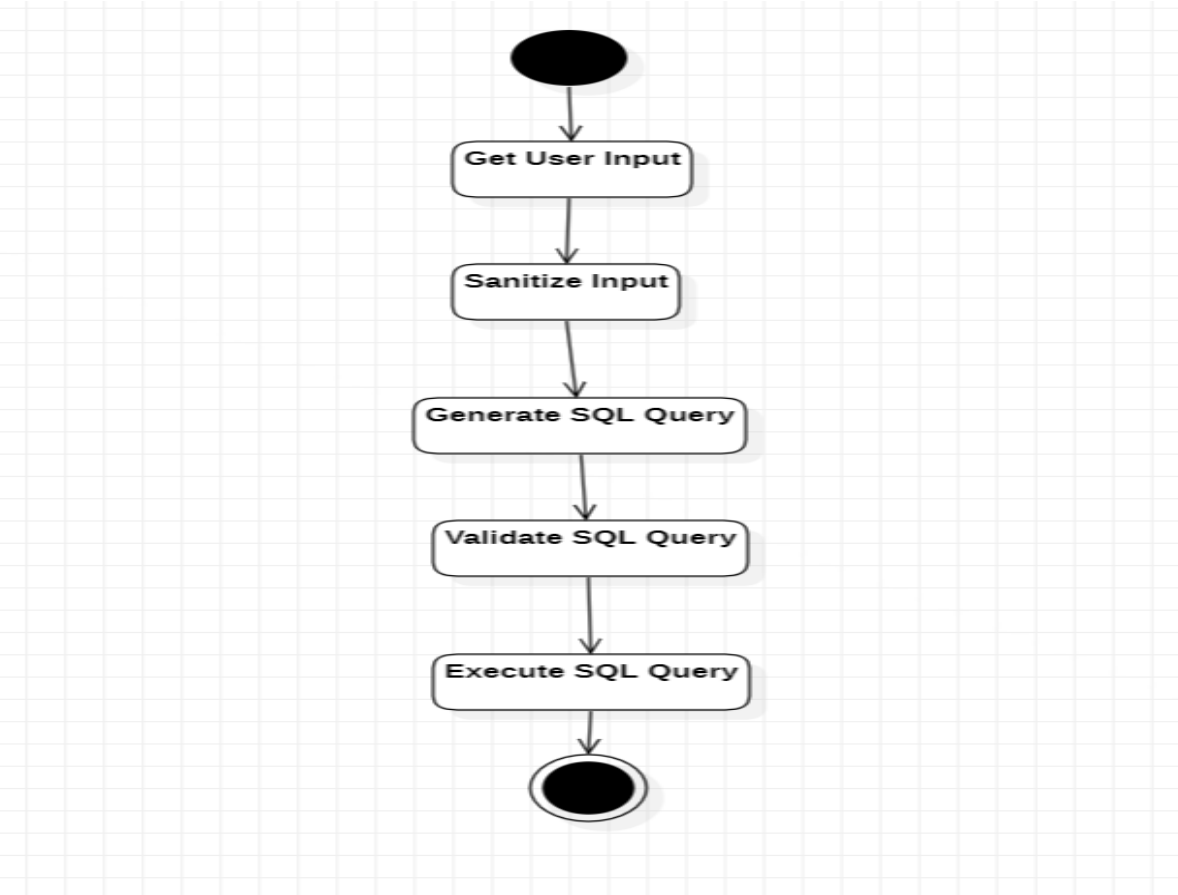


Fig: SQL Injection Attack

Activity Diagram



The Prevent SQL Injection activity starts with the Start node and ends with the End node. In between, the following steps are taken:

Get User Input: This step retrieves user input that will be used in a SQL query.

Sanitize Input: The user input is passed through a sanitization process to remove any potentially harmful characters or strings.

Generate SQL Query: A SQL query is generated based on the sanitized user input.

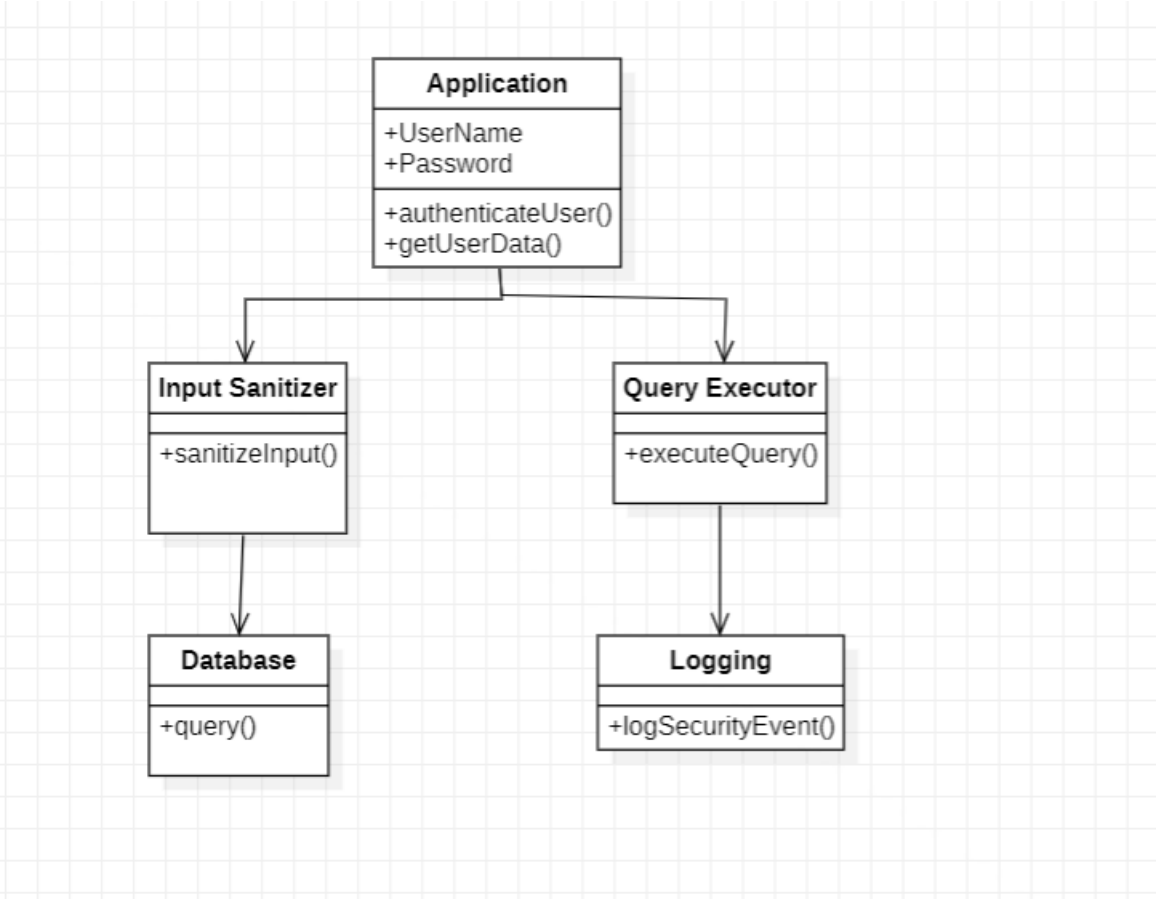
Validate SQL Query: The generated SQL query is validated to ensure it contains no potentially harmful SQL statements.

Execute SQL Query: The validated SQL query is executed against the database.

Note that the Sanitize Input and Validate SQL Query steps are key to preventing SQL injection attacks. By removing potentially harmful characters and validating SQL queries before they are executed, the risk of a successful SQL injection attack is greatly reduced.

This UML activity diagram provides a high-level overview of the steps involved in preventing SQL injection attacks

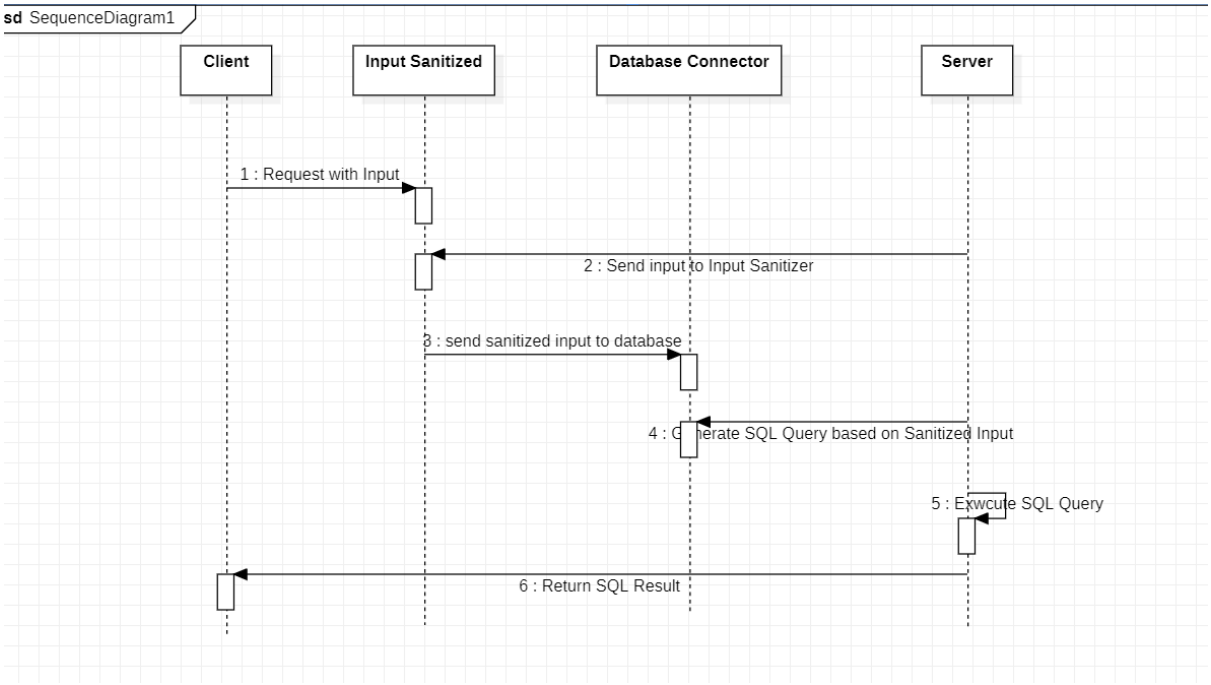
Class Diagram



This diagram illustrates the components involved in SQL injection prevention, which include the Application, Input Sanitizer, Query Executor, Database, and Logging components. The Application component is responsible for authenticating users and retrieving user data. The Input Sanitizer component is responsible for sanitizing user input to prevent SQL injection attacks. The Query Executor component is responsible for executing SQL queries in a safe and secure manner. The Database component stores and retrieves data, while the Logging component logs security events.

The arrows in the diagram represent the flow of control between components. For example, when the Application component calls authenticateUser(), it passes user credentials to the Input Sanitizer component, which sanitizes the input and passes it to the Query Executor component. The Query Executor component then executes a secure SQL query against the Database, retrieves the user data, and passes it back to the Application component. If a security event occurs, such as a SQL injection attempt, the Logging component logs the event.

Sequence Diagram



CHAPTER 5: APPLICATION IMPLEMENTATION

5.1 What is SQL Injection?

SQL injection is a type of security exploit in which the attacker adds Structured Query Language (SQL) code to a Web form input box to gain access to resources or make changes to data. An SQL query is a request for some action to be performed on a database. Typically, on a Web form for user authentication, when a user enters their name and password into the text boxes provided for them, those values are inserted into a SELECT query. If the values entered are found as expected, the user is allowed access; if they aren't found, access is denied.

However, most Web forms have no mechanisms in place to block input other than names and passwords. Unless such precautions are taken, an attacker can use the input boxes to send their own request to the database, which could allow them to download the entire database or interact with it in other illicit ways.

5.2 Why we Preventing SQL Injection?

Preventing SQL injection attacks is essential for the security of any web application that interacts with a database. Here are some general best practices for preventing SQL injection attacks:

- Use prepared statements and parameterized queries: Prepared statements are templates that allow you to bind input parameters to placeholders in SQL statements. This approach makes it much more difficult for attackers to inject malicious SQL code into the query.

- Sanitize user input: Any data that is received from users should be validated and sanitized to ensure that it contains only the expected characters and is not carrying any malicious code.

- Limit database permissions: Ensure that your database user has only the minimum necessary permissions required to perform its tasks. For example, the database user for a web application should not have permissions to delete tables or modify database schemas.

- Use a web application firewall: A web application firewall can detect and block many types of SQL injection attacks, including both known and unknown attacks.

- Keep software up to date: Ensure that all software components used by your web application, including your database, are kept up to date with the latest security patches and updates.

5.3 Types of SQL Injection Attacks

SQL injection is a type of security vulnerability that occurs when an attacker injects malicious SQL code into a vulnerable application's input fields. The attacker can use this vulnerability to access or manipulate sensitive data, execute unauthorized commands, and even take control of the entire database.

Here are some of the most common types of SQL injection attacks:

Classic SQL injection: This is the most basic form of SQL injection. It occurs when an attacker inserts malicious SQL code into an application's input field to gain unauthorized access to sensitive data.

Error-based SQL injection: This type of SQL injection occurs when an attacker exploits an error in the application's SQL code to extract sensitive information.

Union-based SQL injection: This type of SQL injection is similar to classic SQL injection, but it involves using the UNION operator to combine data from multiple tables.

Blind SQL injection: This type of SQL injection occurs when an attacker sends SQL queries to the database, but the application does not display the results of those queries.

Time-based SQL injection: This type of SQL injection uses delays to extract sensitive information from the database. The attacker sends a malicious SQL query that delays the application's response time, allowing them to extract data without being detected.

Out-of-band SQL injection: This type of SQL injection uses a different channel, such as email or DNS, to extract data from the database. The attacker sends a malicious SQL query that triggers a response on a different channel, allowing them to extract data without being detected.

It's important to note that these types of SQL injection attacks can overlap, and attackers often use multiple techniques to maximize their impact.

CHAPTER 6: TESTING

TESTING FOR PERFORMANCE:

After you have identified specific performance requirements, you can begin testing to determine whether the application meets those requirements. Performance testing presumes that the application is functioning, stable and robust. As such it's important to eliminate as many variables as possible. For example, bugs in the code can create the appearance, performance problem or even mask performance problems

TESTING FOR RELIABILITY:

Testing for reliability is about exercising an application so that failures are discovered and removed before the system is deployed because the different combinations of pathways through an application are high; it is unlikely that you can identify all potential failures in complex application.

WHAT IS SQL INJECTION TESTING?

SQL injection tests insert data into your application to verify that you can run user-controlled SQL queries on your database. A test successfully finds a SQL injection vulnerability when a certain user input, which could be used for a malicious input, is accepted by the application without proper validation.

SQL injection testing is critical to protect web applications that rely on databases. Successful exploitation of SQL injection vulnerabilities could allow unauthorized users to access and manipulate data in the database, and in some cases, compromise the web application and server.

6.1 Testing Methods

SQL injection testing requires understanding an application's interaction with a database server to access data. Applications often communicate with a database to authenticate web forms (checking credentials against the database) and perform searches (user-submitted input can extract data from the database via SQL queries). Testers must list the input fields with values that could end up in an SQL query.

Usually, the first test involves adding a quotation mark or semicolon to a parameter or field. If the application doesn't filter the input, the value in quotation marks could produce a flawed query, while the semicolon could produce an error.

Testers can also use SQL keywords (e.g., AND, OR) and comment delimiters like — and /* */ to try modifying a query. They can insert a string in the place of a number to generate an error. The testers should monitor all web server responses and inspect the source code.

The responses may contain an error without presenting it to the user. Proper error messages offer invaluable information, allowing testers to recreate an injection attack successfully.

Unfortunately, applications do not always provide the full details of an error, issuing simple 500 errors. In these cases, testers must use blind SQL injection methods. Testers should check every field individually to identify the vulnerable parameters.

Here are some of the testing methods to help identify SQL vulnerabilities:

1. Stacked Query Testing

In the stacked query method, testers complete an SQL statement and write a new one. Testers and developers should ensure that their applications do not support stacked queries (where possible). For example, developers should avoid using a multi-query statement that enables stacked queries.

2. Error-Based Injection Testing

Error-based injection exploits the SQL error messages displayed to users. Users attempt something that likely causes an error and retrieve data from the error message produced. Users with access to information like the names of tables can more easily compromise the underlying database.

3. Boolean-Based Injection Testing

The Boolean method involves appending conditions to conditional statements (true in some cases, false in others). Attackers can perform several conditional queries to learn about the database. Testers can use this attack method to identify Boolean-based injection vulnerabilities.

To prevent Boolean-based injection attacks, teams must ensure the application never runs user input as SQL code. One way to achieve this is with prepared statements that ensure SQL does not interpret user input as code.

4. Out-of-Band (Blind) Exploit Testing

Out-of-band exploit tests are useful for assessing blind SQL injection vulnerabilities, where the attacker doesn't know anything about the operation's outcome. This method uses Database Management System (DBMS) functions to perform out-of-band connections and deliver query results to the attacker's server.

5. Time Delay Exploit Testing

Time delay exploits are useful for blind SQL injection situations. This method involves sending injected queries and monitoring the server's response time (if the conditional is true). A delayed response indicates that the conditional query's result is true.

6.2 SQL Injection Testing: Manual vs Automated testing

- Before an organization can secure its applications or websites, it is essential to know about any SQL injection vulnerabilities they contain. SQL injection is a popular attack technique that often impacts businesses severely. Testing teams should test application code for SQL injection vulnerabilities regularly.

- Organizations should ideally test their code upon each update. Frequent testing allows security and development teams to identify and address issues introduced in code changes. Testers can look for SQL injection vulnerabilities using manual or automated methods, leveraging scanning tools to accelerate the process.

- Manual SQL injection testing involves manually applying user-supplied inputs to various fields to assess the application or website's input validation. It is often a time-consuming method, especially when testing many fields. Manual techniques may be inadequate to test everything thoroughly. Given the sheer scale of the task, testers can easily overlook some vulnerabilities.

- Organizations often use automated scanning tools to identify SQL injection vulnerabilities, allowing developers to address coding issues. Web security scanning tools offer a fast, comprehensive testing technique, returning detailed results about any vulnerabilities they discover. Testers can more easily identify affected parameters and URLs, saving time and enabling frequent software updates.

6.3 Different Test Cases

SQL injection is a type of web application security vulnerability that allows an attacker to execute malicious SQL statements against a web application's backend database. To prevent SQL injection, it is important to properly validate and sanitize user input.

Here are some test cases you can use to test SQL injection prevention:

1. **Input validation:** Test if the web application properly validates user input by attempting to enter SQL commands in the input fields. For example, try to enter a SQL statement in a username or password field.

2. **Parameterized queries:** Test if the web application uses parameterized queries to prevent SQL injection. For example, try to modify the SQL statement in a parameterized query to include a malicious statement.

3. **Error messages:** Test if the web application provides detailed error messages when an SQL injection attack is attempted. For example, try to enter a SQL statement in the input field and see if the web application provides an error message that indicates that an SQL injection attack was detected.

4. **Stored procedures:** Test if the web application uses stored procedures to prevent SQL injection. For example, try to execute an SQL statement directly against the database and see if the stored procedure prevents it.

5. **Encoding:** Test if the web application properly encodes user input to prevent SQL injection. For example, try to enter a SQL statement using different types of encoding techniques, such as URL encoding or base64 encoding.

6. **White-listing:** Test if the web application uses a white-list approach to input validation. For example, try to enter input that is not on the allowed list and see if the web application rejects it.

7. **Blacklisting:** Test if the web application uses a blacklist approach to input validation. For example, try to enter input that is on the blocked list and see if the web application rejects it.

8. **Out-of-band attacks:** Test if the web application can be vulnerable to out-of-band SQL injection attacks. For example, try to execute a DNS or HTTP request to an attacker-controlled server using the database.

By testing these test cases, you can identify any vulnerabilities and improve the SQL injection prevention mechanisms in your web application.

CHAPTER 7: CODE IMPLEMENTATION

PHP Source Code

WELCOME.PHP

```
<?php
/* Initialize the session */
session_start();
/* Check if the user is logged in, if not then redirect him to login page */
if(!isset($_SESSION["loggedin"]) || $_SESSION["loggedin"] !== true){
    header("location: login.php");
    exit;
}
?>
<!DOCTYPE html>
<html lang="en">
<head>
    <title>Welcome</title>
    <link rel="stylesheet" href="bootstrap.css">
    <style type="text/css">
        body{ font: 14px sans-serif; text-align: center; }
    </style>
</head>
<body>
    <div class="page-header">
        <h1>Hi, <b><?php echo htmlspecialchars($_SESSION["username"]); ?></b>. Welcome</h1>
    </div>
    <p><a href="logout.php" class="btn btn-danger">Sign Out of Your Account</a></p>
</body>
</html>
```

REGISTER.PHP

```
<?php
/* Include config file */
require_once "config.php";

/* Define variables and initialize with empty values */
$username = $password = $confirm_password = "";
$username_err = $password_err = $confirm_password_err = "";

/* Processing form data when form is submitted */
if ($_SERVER["REQUEST_METHOD"] == "POST")
{

   /* Validate username */
   if (empty(trim($_POST["username"])))
   {
      $username_err = "Please enter a username.";
   }
   else
   {
      /* Prepare a select statement */
      $sql = "SELECT id FROM users WHERE username = ?";

      if ($stmt = mysqli_prepare($link, $sql))
      {
         /* Bind variables to the prepared statement as parameters */
         mysqli_stmt_bind_param($stmt, "s", $param_username);

         /* Set parameters */
         $param_username = trim($_POST["username"]);
```

```
        /* Attempt to execute the prepared statement */
        if (mysqli_stmt_execute($stmt))
        {
            /* store result */
            mysqli_stmt_store_result($stmt);

            if (mysqli_stmt_num_rows($stmt) == 1)
            {
                $username_err = "This username is already taken.";
            }
            else
            {
                $username = trim($_POST["username"]);
            }
        }
        else
        {
            echo "Oops! Something went wrong. Please try again later.";
        }

        /* Close statement */
        mysqli_stmt_close($stmt);
    }
}

/* Validate password */
if (empty(trim($_POST["password"])))
{
    $password_err = "Please enter a password.";
}
```

```
elseif (strlen(trim($_POST["password"])) < 6)
{
   $password_err = "Password must have atleast 6 characters.";
}
else
{
   $password = trim($_POST["password"]);
}

/* Validate confirm password */
if (empty(trim($_POST["confirm_password"])))
{
   $confirm_password_err = "Please confirm password.";
}
else
{
   $confirm_password = trim($_POST["confirm_password"]);
   if (empty($password_err) && ($password != $confirm_password))
   {
      $confirm_password_err = "Password did not match.";
   }
}

/* Check input errors before inserting in database */
if (empty($username_err) && empty($password_err) && empty($confirm_password_err))
{

   /* Prepare an insert statement */
   $sql = "INSERT INTO users (username, password) VALUES (?, ?)";

   if ($stmt = mysqli_prepare($link, $sql))
```

```
        {
            /* Bind variables to the prepared statement as parameters */
            mysqli_stmt_bind_param($stmt, "ss", $param_username, $param_password);

            /* Set parameters */
            $param_username = $username;
            $param_password = $password;
            /* Creates a password hash
            Attempt to execute the prepared statement */
            if (mysqli_stmt_execute($stmt))
            {
                /* Redirect to login page */
                header("location: login.php");
            }
            else
            {
                echo "Something went wrong. Please try again later.";
            }

            /* Close statement */
            mysqli_stmt_close($stmt);
        }
    }

    /* Close connection */
    mysqli_close($link);
}
?>

<!DOCTYPE html>
<html lang="en">
```

```
<head>
    <meta charset="UTF-8">
    <title>Sign Up</title>
    <link rel="stylesheet" href="bootstrap.css">
    <style type="text/css">
        body{ font: 14px sans-serif; }
        .wrapper{ width: 350px; padding: 20px; }
    </style>
</head>
<body>
    <div class="wrapper">
        <h2>Sign Up</h2>
        <p>Please fill this form to create an account.</p>
        <form action="<?php echo htmlspecialchars($_SERVER["PHP_SELF"]); ?>" method="post">
            <div class="form-group <?php echo (!empty($username_err)) ? 'has-error' : ''; ?>">
                <label>Username</label>
                <input type="text" name="username" autocomplete="off" class="form-control"
value="<?php echo $username; ?>">
                <span class="help-block"><?php echo $username_err; ?></span>
            </div>
            <div class="form-group <?php echo (!empty($password_err)) ? 'has-error' : ''; ?>">
                <label>Password</label>
                <input type="password" name="password" autocomplete="off" class="form-control"
value="<?php echo $password; ?>">
                <span class="help-block"><?php echo $password_err; ?></span>
            </div>
            <div class="form-group <?php echo (!empty($confirm_password_err)) ? 'has-error' : ''; ?>">
                <label>Confirm Password</label>
                <input type="password" name="confirm_password" autocomplete="off" class="form-
control" value="<?php echo $confirm_password; ?>">
                <span class="help-block"><?php echo $confirm_password_err; ?></span>
            </div>
```

```
        <div class="form-group">
            <input type="submit" class="btn btn-primary" value="Submit">
            <input type="reset" class="btn btn-default" value="Reset">
        </div>
        <p>Already have an account? <a href="login.php">Login here</a>.</p>
    </form>
  </div>
</body>
</html>
```

LOGIN.PHP

```
<?php
/* Include config file */
require_once "config.php";

/* Define variables and initialize with empty values */
$username = $password = $confirm_password = "";
$username_err = $password_err = $confirm_password_err = "";

/* Processing form data when form is submitted */
if ($_SERVER["REQUEST_METHOD"] == "POST")
{

  /* Validate username */
  if (empty(trim($_POST["username"])))
  {
    $username_err = "Please enter a username.";
  }
  else
  {
```

```
/* Prepare a select statement */
$sql = "SELECT id FROM users WHERE username = ?";

if ($stmt = mysqli_prepare($link, $sql))
{
   /* Bind variables to the prepared statement as parameters */
   mysqli_stmt_bind_param($stmt, "s", $param_username);

   /* Set parameters */
   $param_username = trim($_POST["username"]);

   /* Attempt to execute the prepared statement */
   if (mysqli_stmt_execute($stmt))
   {
      /* store result */
      mysqli_stmt_store_result($stmt);

      if (mysqli_stmt_num_rows($stmt) == 1)
      {
         $username_err = "This username is already taken.";
      }
      else
      {
         $username = trim($_POST["username"]);
      }
   }
   else
   {
      echo "Oops! Something went wrong. Please try again later.";
   }
```

```
        /* Close statement */
        mysqli_stmt_close($stmt);
    }
}

/* Validate password */
if (empty(trim($_POST["password"])))
{
    $password_err = "Please enter a password.";
}
elseif (strlen(trim($_POST["password"])) < 6)
{
    $password_err = "Password must have atleast 6 characters.";
}
else
{
    $password = trim($_POST["password"]);
}

/* Validate confirm password */
if (empty(trim($_POST["confirm_password"])))
{
    $confirm_password_err = "Please confirm password.";
}
else
{
    $confirm_password = trim($_POST["confirm_password"]);
    if (empty($password_err) && ($password != $confirm_password))
    {
        $confirm_password_err = "Password did not match.";
    }
```

```
}

/* Check input errors before inserting in database */
if (empty($username_err) && empty($password_err) && empty($confirm_password_err))
{

   /* Prepare an insert statement */
   $sql = "INSERT INTO users (username, password) VALUES (?, ?)";

   if ($stmt = mysqli_prepare($link, $sql))
   {
      /* Bind variables to the prepared statement as parameters */
      mysqli_stmt_bind_param($stmt, "ss", $param_username, $param_password);

      /* Set parameters */
      $param_username = $username;
      $param_password = $password;
      /* Creates a password hash
      Attempt to execute the prepared statement */
      if (mysqli_stmt_execute($stmt))
      {
         /* Redirect to login page */
         header("location: login.php");
      }
      else
      {
         echo "Something went wrong. Please try again later.";
      }
      /* Close statement */
      mysqli_stmt_close($stmt);
   }
```

```
    }
    /* Close connection */
    mysqli_close($link);
}
?>
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>Sign Up</title>
    <link rel="stylesheet" href="bootstrap.css">
    <style type="text/css">
        body{ font: 14px sans-serif; }
        .wrapper{ width: 350px; padding: 20px; }
    </style>
</head>
<body>
    <div class="wrapper">
        <h2>Sign Up</h2>
        <p>Please fill this form to create an account.</p>
        <form action="<?php echo htmlspecialchars($_SERVER["PHP_SELF"]); ?>" method="post">
            <div class="form-group <?php echo (!empty($username_err)) ? 'has-error' : ''; ?>">
                <label>Username</label>
                <input type="text" name="username" autocomplete="off" class="form-control"
value="<?php echo $username; ?>">
                <span class="help-block"><?php echo $username_err; ?></span>
            </div>
            <div class="form-group <?php echo (!empty($password_err)) ? 'has-error' : ''; ?>">
                <label>Password</label>
                <input type="password" name="password" autocomplete="off" class="form-control"
value="<?php echo $password; ?>">
                <span class="help-block"><?php echo $password_err; ?></span>
```

```
        </div>
        <div class="form-group <?php echo (!empty($confirm_password_err)) ? 'has-error' : ''; ?>">
            <label>Confirm Password</label>
            <input type="password" name="confirm_password" autocomplete="off" class="form-control" value="<?php echo $confirm_password; ?>">
            <span class="help-block"><?php echo $confirm_password_err; ?></span>
        </div>
        <div class="form-group">
            <input type="submit" class="btn btn-primary" value="Submit">
            <input type="reset" class="btn btn-default" value="Reset">
        </div>
        <p>Already have an account? <a href="login.php">Login here</a>.</p>
    </form>
  </div>
</body>
</html>
```

LOGOUT.PHP

```
<?php
/* Initialize the session */
session_start();
/* Unset all of the session variables */
$_SESSION = array();
/* Destroy the session */
session_destroy();
/* Redirect to login page */
header("location: login.php");
exit;
?>
```

CONFIG.PHP

```
<?php
/* Database credentials. Assuming you are running MySQL
server with default setting (user 'root' with no password) */

define('DB_SERVER', 'localhost');
define('DB_USERNAME', 'root');
define('DB_PASSWORD', '');
define('DB_NAME', 'Sql_injection');

/* Attempt to connect to MySQL database */
$link = mysqli_connect(DB_SERVER, DB_USERNAME, DB_PASSWORD, DB_NAME);

/* Check connection */
if($link === false){
    die("ERROR: Could not connect. " . mysqli_connect_error());
}
?>
```

Creating the database table

```
CREATE TABLE users (
    id INT NOT NULL PRIMARY KEY AUTO_INCREMENT,
    username VARCHAR(50) NOT NULL UNIQUE,
    password VARCHAR(255) NOT NULL,
    created_at DATETIME DEFAULT CURRENT_TIMESTAMP
);
```

STYLE1.CSS

```
@import
url('https://fonts.googleapis.com/css2?family=Poppins:wght@200;300;400;500;600;700&display=sw
ap');
*{
  margin: 0;
  padding: 0;
  box-sizing: border-box;
  font-family: 'Poppins',sans-serif;
}
body{
  height: 100vh;
  display: flex;
  justify-content: center;
  align-items: center;
  padding: 10px;
  background: linear-gradient(135deg, #71b7e6, #9b59b6);
}
.container{
  max-width: 700px;
  width: 100%;
  background-color: #fff;
  padding: 25px 30px;
  border-radius: 5px;
  box-shadow: 0 5px 10px rgba(0,0,0,0.15);
}
.container .title{
  font-size: 25px;
  font-weight: 500;
  position: relative;
}
```

```
.container .title::before{
  content: "";
  position: absolute;
  left: 0;
  bottom: 0;
  height: 3px;
  width: 30px;
  border-radius: 5px;
  background: linear-gradient(135deg, #71b7e6, #9b59b6);
}
.content form .user-details{
  display: flex;
  flex-wrap: wrap;
  justify-content: space-between;
  margin: 20px 0 12px 0;
}
form .user-details .input-box{
  margin-bottom: 15px;
  width: calc(100% / 2 - 20px);
}
form .input-box span.details{
  display: block;
  font-weight: 500;
  margin-bottom: 5px;
}
.user-details .input-box input{
  height: 45px;
  width: 100%;
  outline: none;
  font-size: 16px;
  border-radius: 5px;
```

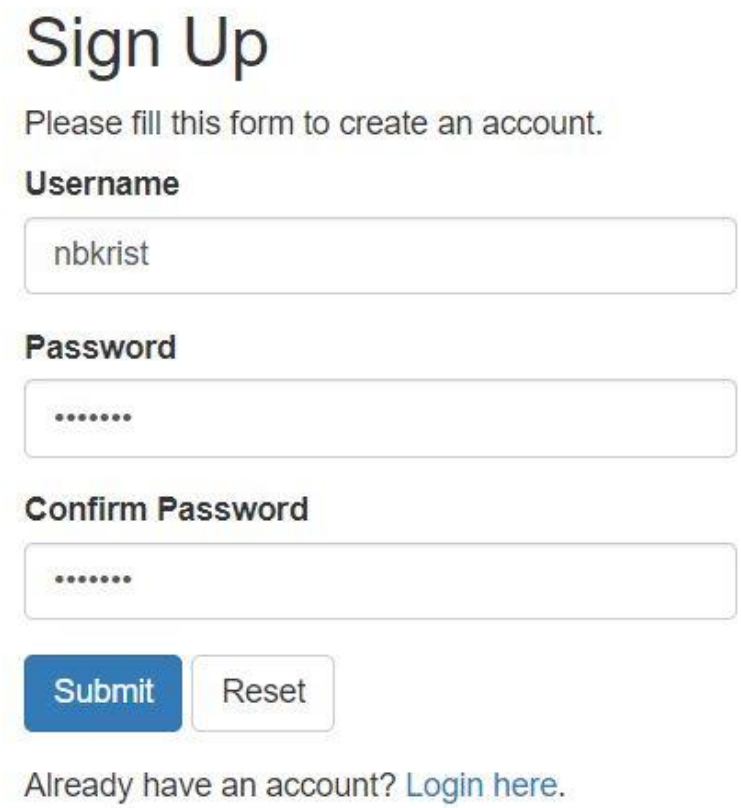
```
  padding-left: 15px;
  border: 1px solid #ccc;
  border-bottom-width: 2px;
  transition: all 0.3s ease;
}
.user-details .input-box input:focus,
.user-details .input-box input:valid{
  border-color: #9b59b6;
}
 form .gender-details .gender-title{
  font-size: 20px;
  font-weight: 500;
 }
 form .category{
   display: flex;
   width: 80%;
   margin: 14px 0 ;
   justify-content: space-between;
 }
 form .category label{
   display: flex;
   align-items: center;
   cursor: pointer;
 }
 form .category label .dot{
  height: 18px;
  width: 18px;
  border-radius: 50%;
  margin-right: 10px;
  background: #d9d9d9;
  border: 5px solid transparent;
```

```
 transition: all 0.3s ease;
}
#dot-1:checked ~ .category label .one,
#dot-2:checked ~ .category label .two,
#dot-3:checked ~ .category label .three{
  background: #9b59b6;
  border-color: #d9d9d9;
}
form input[type="radio"]{
  display: none;
}
form .button{
  height: 45px;
  margin: 35px 0
}
form .button input{
  height: 100%;
  width: 100%;
  border-radius: 5px;
  border: none;
  color: #fff;
  font-size: 18px;
  font-weight: 500;
  letter-spacing: 1px;
  cursor: pointer;
  transition: all 0.3s ease;
  background: linear-gradient(135deg, #71b7e6, #9b59b6);
}
form .button input:hover{
 /* transform: scale(0.99); */
 background: linear-gradient(-135deg, #71b7e6, #9b59b6);
```

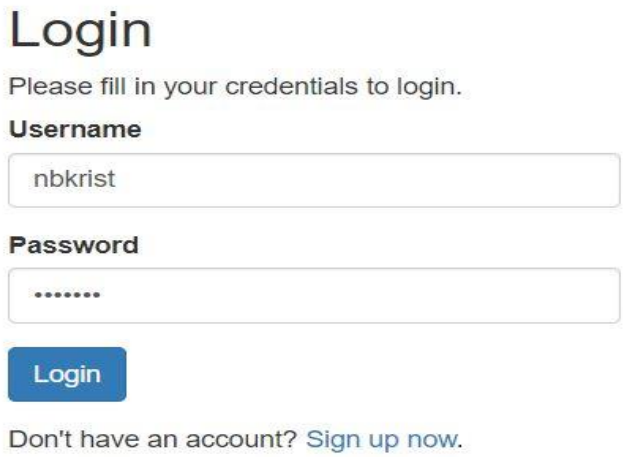
```
  }
 @media(max-width: 584px){
 .container{
  max-width: 100%;
}
form .user-details .input-box{
    margin-bottom: 15px;
    width: 100%;
  }
  form .category{
    width: 100%;
  }
  .content form .user-details{
    max-height: 300px;
    overflow-y: scroll;
  }
  .user-details::-webkit-scrollbar{
    width: 5px;
  }
  }
  @media(max-width: 459px){
  .container .content .category{
    flex-direction: column;
  }
}
```

CHAPTER 8: USER MANUAL

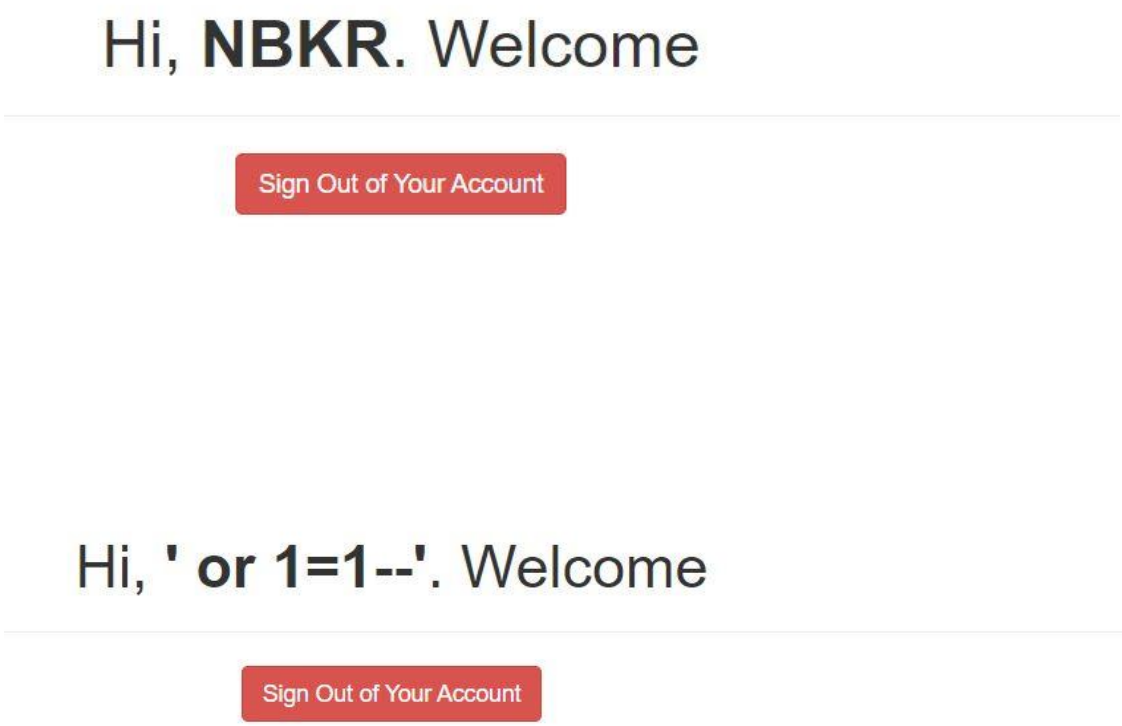
Sign Up Page



Login Page



Welcome Page



CHAPTER 9: CONCLUSION & FUTURE ENHANCEMENTS

9.1 CONCLUSION

SQL injection attackers are smarter and more comprehensive in finding vulnerable websites. There are some new methods of SQL attack. Hackers can use various tools to speed up the process of exploiting vulnerabilities. Let us look at the Asprox Trojan, which is spread mainly through a botnet that publishes mail. The whole process of its work can be described as follows: First, the Trojan is installed on the computer through spam sent by the controlled host. Then, the computer infected by the Trojan will download a binary code, and when it starts, it will use search. Index engine search uses Microsoft's ASP technology to build forms and vulnerable websites. The results of the search become a list of targets for SQL injection attacks. Then, the Trojan Horse will launch SQL injection attacks on these sites, making some sites under ontrol and destruction. Users visiting these controlled and destroyed sites will be tricked into downloading a malicious piece of JavaScript code from another site. Finally, this code guides users to the third site, where there are more malware, such as Trojan horses stealing passwords.

9.2 FUTURE ENHANCEMENT

In the future, additional enhancements to SQL injection prevention may include:

- **Machine Learning-Based Detection:** Machine learning algorithms can be used to analyze user input and identify patterns that are indicative of SQL injection attacks. By using machine learning, web applications can proactively detect and prevent SQL injection attacks before they occur.

- **Real-Time Monitoring:** Real-time monitoring involves continuously monitoring user input and database activity for suspicious behavior. By using real-time monitoring, web applications can quickly identify and respond to SQL injection attacks as they occur.

- **Database Firewalls:** Database firewalls can be used to monitor and filter incoming traffic to the database. By using a database firewall, you can block malicious traffic and prevent SQL injection attacks from reaching the database.

- **Automatic Patching:** Automatic patching involves automatically applying security updates and patches to web applications and databases. By using automatic patching, web applications can stay up-to-date with the latest security fixes and prevent known vulnerabilities from being exploited.

9.3 REFERENCE

[1] Shen Qingni, Qingsi. Operating system security design. Beijing: Machinery Industry Press, 2013.

[2] Yu Chaohui, Wang Changzheng, Zhao Yicheng. Practical Treasure Book of Network Security System Protection and Hacker Attack and Defense. Beijing: China Railway Publishing House, 2013.

[3] Ma Limei, Wang Fangwei. Computer Network Security and Experimental Course, tsinghua university press,ISBN:9787302439332

[4] Ma Limei,GuoQing,ZhangLinwei Ubuntu Linux operating system and Experimental Course, tsinghua university press,ISBN:9787302438236

[5] Dong Zhenliang. Application of cryptographic algorithms and international standardization [D]. Financial Information Center of the People's Bank of China, 2018.

[6] Zhou Yinqing, Ouyang Zichun. Brief discussion on the implementation and management of information system security level protection evaluation [D]. Digital Communication World, 2018.

[7] Liang Lixin and Li Jun. Information Security Level Protection Evaluation Based on Virtualization [D]. Police Technology, 2014

[8] Wubin,Liu Dun. SQL Injection Attack and Vulnerability Detection Network Security Technology & Application,2017

9.4 WEBSITES

- https://github.com/rkhal101/Web-Security-Academy-Series/tree/main/sql-injection/theory
- https://phrack.org/issues/54/8.html
- https://www.alertlogic.com/blog/tried-and-true-sql-injection-still-a-leading-method-of-cyber-attack-d58
- https://www.owasp.orgjindex.php/Top 10 2013-Top 10